

```

-----
todocrud-----
import {
  addCurso,
  getCursosCollection,
  deleteCursoCollection,
  getCursoCollection,
  updateCursoCollection,
  getPreceptoresCollection,
} from "../firebase.js";

/**
 * Función para levantar Venta Modal
 */
window.miModal = async function (idModal, idCurso = "") {
  try {
    await validarModal(idModal);

    let url = "";
    switch (idModal) {
      case "agregarCursoModal":
        url = "modales/modalAddCurso.php";
        break;
      case "detalleCursoModal":
        url = "modales/modalDetallesCurso.php";
        break;
      case "editarCursoModal":
        url = "modales/modalEditarCurso.php";
        break;
      case "eliminarCursoModal":
        url = "modales/modalDeleteCurso.php";
        break;
      default:
        throw new Error(`El idModal '${idModal}' no es válido`);
    }

    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Error al cargar la modal");
    }

    // response.text() es un método en programación que se utiliza para obtener el
    contenido de texto de una respuesta HTTP
    const data = await response.text();

    // Crear un elemento div para almacenar el contenido de la modal
    const modalContainer = document.createElement("div");
    modalContainer.innerHTML = data;
  }
};

```

```

// Agregar la modal al documento actual
document.body.appendChild(modalContainer);

// Mostrar la modal
const modalElement = modalContainer.querySelector(`#${idModal}`);
const myModal = new bootstrap.Modal(modalElement);
myModal.show();
if (idModal === "agregarCursoModal") { // Ajusta al ID real de tu modal de curso
  await cargarPreceptoresEnModal();
} else if (idModal === "agregaralumnoModal") {
  await cargarCursosEnModal();
}
  if (idModal === "detalleCursoModal") {
    await cargarDetalleCurso(idCurso);
  } else if (idModal === "editarCursoModal") {
    await getCursoUpdateCollection(idCurso);
  } else if (idModal === "eliminarCursoModal") {
    let DeleteBtn = document.querySelector("#confirmDeleteBtn");
    DeleteBtn.addEventListener("click", async () => {
      await eliminarCurso(idCurso);
    });
  }
} catch (error) {
  console.error(error);
}
};

//Función para validar si existe una modal abierta
async function validarModal(idModal) {
  const existingModal = document.getElementById(idModal);
  if (existingModal) {
    const modal = bootstrap.Modal.getInstance(existingModal);
    if (modal) {
      modal.hide();
    }
    existingModal.remove();
  }
}

/**
 * Función para obtener todas las colecciones
 */
async function mostrarCursosEnHTML() {
  try {
    const snapshot = await getCursosCollection(); // Llamada a RTDB
    const tablaCursos = document.querySelector("#tablaCursos tbody");
    tablaCursos.innerHTML = "";

    if (snapshot.exists()) {
      // RTDB usa forEach sobre el snapshot directamente

```

```

    snapshot.forEach((childSnapshot) => {
        const id = childSnapshot.key; // El ID generado por push()
        const Curso = childSnapshot.val(); // AQUÍ: usamos .val() en lugar de
.data()

        const fila = document.createElement("tr");
        fila.id = id;
        fila.innerHTML = `
            <td>${Curso.curso || "N/A"}</td>
            <td>${Curso.ubicacion || "N/A"}</td>
            <td>${Curso.capacidad || "N/A"}</td>
            <td>${Curso.obs || "N/A"}</td>
            <td>
                <button onclick="window.miModal('detalleCursoModal','${id}')"
class="btn btn-success"><i class="bi bi-binoculars"></i></button>
                <button onclick="window.miModal('editarCursoModal','${id}')" class="btn
btn-warning"><i class="bi bi-pencil-square"></i></button>
                <button onclick="window.miModal('eliminarCursoModal','${id}')"
class="btn btn-danger"><i class="bi bi-trash"></i></button>
            </td>
        `;
        tablaCursos.appendChild(fila);
    });
}
} catch (error) {
    console.error("Error al obtener los Cursos:", error);
}
}

window.addEventListener("DOMContentLoaded", mostrarCursosEnHTML);

/**
 * Función para agregar un nuevo Curso
 */

window.addNuevoCurso = async function (event) {
    event.preventDefault();

    const formulario = document.querySelector("#formularioCursoEdit");
    const preceptorId = document.getElementById("selectPreceptorModal").value;
    if (!formulario) return;

    // Extraemos los datos del formulario
    const formData = new FormData(formulario);
    const curso = formData.get("curso");
    const ubicacion = formData.get("ubicacion");
    const capacidad = formData.get("capacidad");
    const obsCurso = formData.get("obsCurso");
    const preceptorId = formData.get("preceptorId"); // Esto busca el atributo
'name' del select

```

```

// Debug para ver qué se está capturando
console.log("Datos capturados:", { curso, preceptorId });

if (!preceptorId || preceptorId === "") {
  alert("Por favor, seleccione un preceptor");
  return;
}

try {
  // Llamada a la función de firebase.js
  await addCurso(curso, ubicacion, capacidad, obsCurso, preceptorId);

  formulario.reset();

  // Cerrar modal (usando Bootstrap 5 nativo ya que tienes errores con
jQuery)
  const modalElt = document.getElementById('agregarCursoModal');
  const modal = bootstrap.Modal.getInstance(modalElt);
  if (modal) modal.hide();

  await mostrarCursosEnHTML();
  window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso creado con
éxito!" });
} catch (error) {
  console.error("Error al crear el curso:", error);
  window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al crear el
curso" });
}
};

```

```

/**
 * Función para cargar y mostrar los detalles del Curso en la modal
 */
async function cargarDetalleCurso(id) {
  try {
    const snapshot = await getCursoCollection(id); // Calling the fixed function
above

    if (snapshot.exists()) {
      const CursoData = snapshot.val(); // USE .val() instead of .data()
      const { curso, ubicacion, capacidad, obs } = CursoData;

      const ulDetalleCurso = document.querySelector("#detalleCursoContenido ul");
      ulDetalleCurso.innerHTML = `
        <li class="list-group-item"><b>Curso:    </b> ${curso    || "No

```

```

disponible"></li>
    <li class="list-group-item"><b>Ubicación:</b> ${ubicacion || "No
disponible"></li>
    <li class="list-group-item"><b>Capacidad:</b> ${capacidad || "No
disponible"></li>
    <li class="list-group-item"><b>Obs:      </b> ${obs      || "No
disponible"></li>
    `;
  } else {
    console.log("No se encontró ningún Curso con el ID:", id);
  }
} catch (error) {
  console.error("Error al mostrar detalles del Curso", error);
}
}

```

```

/**
 * Buscar Curso a editar
 */
async function getCursoUpdateCollection(id) {
  try {
    const CursoDoc = await getCursoCollection(id);
    if (CursoDoc.exists()) {
      const CursoData = CursoDoc.val();
      const { curso, ubicacion, capacidad, obs } = CursoData;
      document.querySelector("#idCurso").value = id;
      document.querySelector("#curso").value = curso;
      document.querySelector("#ubicacion").value = ubicacion;
      document.querySelector("#capacidad").value = capacidad;
      document.querySelector("#obsCurso").value = obs;
    } else {
      console.log("No se encontró ningún Curso con el ID:", id);
    }
  } catch (error) {
    console.error("Error al obtener los detalles del Curso:", error);
  }
}

```

```

/**
 * Función para actualizar el Curso
 */
window.actualizarCurso = async function (event) {
  event.preventDefault();
  const formulario = document.querySelector("#formularioCursoEdit");
  const formData = new FormData(formulario);

  // Convertir FormData a un objeto JSON
  const formDataJSON = {};
  formData.forEach((value, key) => {

```

```

        //console.log(key, value);
        formDataJSON[key] = value;
    });

    const { idCurso, curso, ubicacion, capacidad, obsCurso } = formDataJSON;
    try {
        await updateCursoCollection(idCurso, { curso, ubicacion, capacidad,
obs:obsCurso});
        formulario.reset();

        setTimeout(() => {
            $("#editarCursoModal").css("opacity", "");
            $("#editarCursoModal").modal("hide");
        }, 300);

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso actualizado
correctamente!" });
        await mostrarCursosEnHTML();
    } catch (error) {
        console.log(error);
    }
};

```

```

/**
 * Función para borrar un Curso, una colleccion
 */
async function eliminarCurso(id) {
    try {
        await deleteCursoCollection(id);
        document.querySelector(`#${id}`).remove();
        mostrarAlerta({ tipoToast: "success", mensaje: "Curso eliminado correctamente"
});
    } catch (error) {
        console.error("Error al borrar el Curso:", error);
        mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar el Curso" });
    }
}

/**
 * Función para mostrar alertas
 */
iziToast.settings({
    timeout: 10000,

```

```

    resetOnHover: true,
    // icon: '', // icon class
    transitionIn: "flipInX",
    transitionOut: "flipOutX",
    position: "topRight", // bottomRight, bottomLeft, topRight, topLeft, topCenter,
    bottomCenter, center
    onOpening: function () {
        console.log("Alerta abierta!");
    },
    onClosing: function () {
        console.log("Alerta cerrada!");
    },
});
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (tipoToast == "success") {
        iziToast.success({
            timeout: 5000,
            icon: tipoToast == "success" ? "bi bi-check-circle-fill" : "bi
bi-x-circle-fill",
            title: tipoToast == "success" ? "¡Éxito!" : "¡Error!",
            message: mensaje,
        });
    } else if (tipoToast == "warning") {
        iziToast.success({
            timeout: 5000,
            icon: tipo == "success" ? "bi bi-check-circle-fill" : "bi bi-x-circle-fill",
            title: tipo == "success" ? "¡Éxito!" : "¡Error!",
            message: mensaje,
        });
    }
};

```

```

async function cargarPreceptoresEnModal() {
    const select = document.getElementById("selectPreceptorModal");
    if (!select) return;

```

```

    try {
        const preceptorSnap = await getPreceptoresCollection();
        select.innerHTML = '<option selected value="">Selecione
Preceptor</option>';

```

```

        if (preceptorSnap.exists()) {
            preceptorSnap.forEach(doc => {
                const data = doc.val();
                const option = document.createElement("option");

```

```

                // IMPORTANTE: El value ahora es el ID (la clave de Firebase)
                option.value = doc.key;
                // Lo que ve el usuario sigue siendo el nombre
                option.textContent = data.nombre;

```

```

        select.appendChild(option);
    });
}
} catch (error) {
    console.error("Error:", error);
}
}

----- modaladdCurso
-----
<div class="modal fade" id="agregarCursoModal" data-bs-backdrop="static"
data-bs-keyboard="false" tabindex="-1" aria-labelledby="staticBackdropLabel"
aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">
            <div class="modal-header">
                <h1 class="modal-title fs-5 titulo_modal">Agregar Nuevo
Curso</h1>
                <button type="button" class="btn-close" data-bs-dismiss="modal"
aria-label="Close"></button>
            </div>

<div class="modal-body">
        <form id="formularioCursoEdit" action="" method="POST"
autocomplete="off">
            <input type="hidden" name="idCurso" id="idCurso" />
            <div class="mb-3">
                <label class="form-label">Curso</label>
                <input type="text" name="curso" id="curso"
class="form-control" required />
            </div>
            <div class="mb-3">
                <label class="form-label">Seleccione Preceptor</label>
                <select name="preceptorID" id="selectPreceptorModal"
class="form-select" required>
                    <option selected value="">Seleccione</option>
                </select>
            </div>

            <div class="mb-3">
                <label class="form-label">Ubicación</label>
                <input type="text" name="ubicacion" id="ubicacion"
class="form-control" required />
            </div>
            <div class="mb-3">
                <label class="form-label">Capacidad</label>
                <input type="text" name="capacidad" id="capacidad"
class="form-control" required />
            </div>

```

```

        <div class="mb-3">
            <label class="form-label">Obs</label>
            <input type="text" name="obsCurso" id="obsCurso"
class="form-control" required />
        </div>

        <div class="d-grid gap-2">
            <button type="submit" class="btn btn-primary btn_add"
onclick="window.addNuevoCurso(event)">
                Crear Nuevo Curso
            </button>
        </div>
    </form>
</div>

```

```

    </div>
</div>
</div>

```

```

----- firebase
-----
// Configuración de Firebase
// Importante las versiones para 'firebase-app.js' y para 'firebase-firestore.js'
en ambos casos debes ser las mismas
import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.13.0/firebase-app.js";
import {
    getDatabase,
    ref,
    push,
    get,
    child,
    update,
    remove
} from "https://www.gstatic.com/firebasejs/10.13.0/firebase-database.js";

// Firebase configuration (replace with your actual project details)
const firebaseConfig = {
    apiKey: "AIzaSyAfpEDzd8wc6t9Y3foI2HDrWVL_MIzhYnA",
    authDomain: "asistencia-93328.firebaseio.com",
    databaseURL: "https://asistencia-93328-default-rtdb.firebaseio.com",
    projectId: "asistencia-93328",
    storageBucket: "asistencia-93328.firebaseio.com",
    messagingSenderId: "692275978617",
    appId: "1:692275978617:web:5579ef6c0aeb2d58c7cfa8"
};

```

```
// Inicializar Firebase

export const app = initializeApp(firebaseConfig);
export const db = getDatabase(app);
const coleccion = "cursos";
const coleccionPreceptores = "preceptor"; // Nombre de tu tabla en RTDB

// Agrega un curso
export const addCurso = (curso, ubicacion, capacidad, obs, preceptorID) =>
  push(ref(db, coleccion), { curso, ubicacion, capacidad, obs, preceptorID });

// Obtiene todos los cursos
export const getCursosCollection = () => get(ref(db, coleccion));

// Obtiene un curso específico por ID
export const getCursoCollection = (id) => get(ref(db, `${coleccion}/${id}`));

// Actualiza un curso
export const updateCursoCollection = (id, newFields) =>
  update(ref(db, `${coleccion}/${id}`), newFields);

// Elimina un curso
export const deleteCursoCollection = (id) =>
  remove(ref(db, `${coleccion}/${id}`));

export const getPreceptoresCollection = () => get(ref(db, coleccionPreceptores));
```